

Experiment No: 06

Experiment Name: Apply 2D transformation techniques to a triangle (GL-LINES).

Theory:

2D transformations are used in computer graphics to modify the position, size, and orientation of objects in a plane. They help in manipulating shapes such as triangles, rectangles, and polygons. The main transformations include:

- Translation moves an object from one position to another using translation factors t_x and t_y .
- Scaling changes the size of an object using scale factors S_x and S_y .
- Rotation turns an object about the origin or a specific point by an angle θ .
- Reflection produces a mirror image of the object about an axis (x-axis or y-axis).
- Shearing distorts the shape by shifting its coordinates in the x or y direction.

Each of these transformations is represented mathematically using homogeneous coordinates and transformation matrices, which are applied to the object's vertices to obtain the new shape.

Code: Translation ($t_x = 2$, $t_y = 3$).

```
1 #include <GL/glut.h>
2
3 struct Point { float x, y; };
4 Point triangle[3] = {{0,0},{1,0},{0.5,1}};
5
6 void drawTriangle(Point pts[3]) {
7     glBegin(GL_LINES);
8     for(int i=0; i<3; i++){
9         int next = (i+1)%3;
10         glVertex2f(pts[i].x, pts[i].y);
11         glVertex2f(pts[next].x, pts[next].y);
12     }
13     glEnd();
14 }
15
```

```

16 void drawAxes(){
17     glLineWidth(2);
18     glColor3f(1,0,0);
19     glBegin(GL_LINES);
20     glVertex2f(-10,0);
21     glVertex2f(10,0);
22     glEnd();
23
24     glColor3f(0,1,0);
25     glBegin(GL_LINES);
26     glVertex2f(0,-10);
27     glVertex2f(0,10);
28     glEnd();
29     glLineWidth(1);
30 }
31
32 void display() {
33     glClear(GL_COLOR_BUFFER_BIT);
34     drawAxes();
35
36     glColor3f(1,1,1);
37     drawTriangle(triangle);
38
39     Point translated[3];
40     for(int i=0; i<3; i++){
41         translated[i].x = triangle[i].x + 2;
42         translated[i].y = triangle[i].y + 3;
43     }
44
45     glColor3f(1,0,0);
46     drawTriangle(translated);
47
48     glFlush();
49 }
50
51 void init() {
52     glClearColor(0,0,0,1);
53     glMatrixMode(GL_PROJECTION);
54     glLoadIdentity();
55     gluOrtho2D(-10,10,-10,10);
56 }
57
58 int main(int argc, char** argv){
59     glutInit(&argc,argv);
60     glutInitWindowSize(800,800);
61     glutCreateWindow("Translation");
62     init();

```

```
63  glutDisplayFunc(display);
64  glutMainLoop();
65  return 0;
66 }
```

Output:

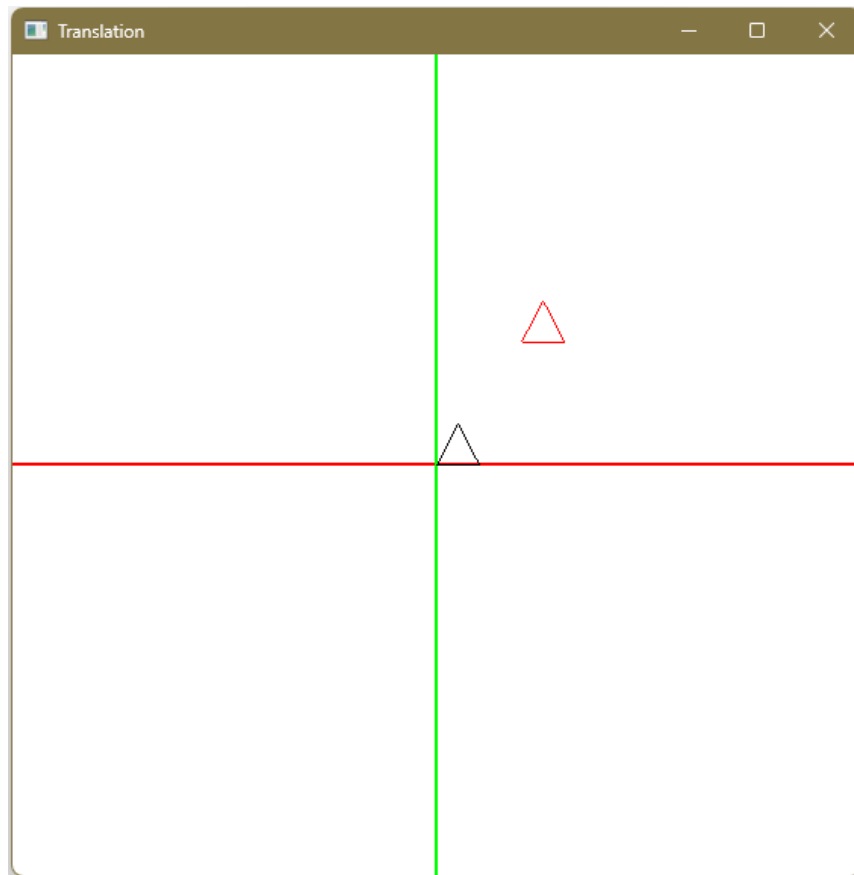


Figure 1: Output of the following code.

Code: Scaling ($S_x = 2$, $S_y = 2$ and $S_x = 0.5$, $S_y = 0.5$).

```
1  #include <GL/glut.h>
2  #include <cmath>
```

```

3
4 struct Point { float x, y; };
5 Point triangle[3] = {{0,0},{1,0},{0.5f,1}};
6
7 void drawTriangle(Point pts[3]) {
8     glBegin(GL_LINES);
9     for(int i=0;i<3;i++){ int n=(i+1)%3; glVertex2f(pts[i].x,pts[i].y); glVertex2f(pts[n].x,pts[n].y); }
10    glEnd();
11 }
12
13 void drawAxes(){
14     glLineWidth(2);
15     glColor3f(1,0,0); glBegin(GL_LINES); glVertex2f(-10,0); glVertex2f(10,0); glEnd();
16     glColor3f(0,1,0); glBegin(GL_LINES); glVertex2f(0,-10); glVertex2f(0,10); glEnd();
17     glLineWidth(1);
18 }
19
20 void display(){
21     glClear(GL_COLOR_BUFFER_BIT);
22     drawAxes();
23
24     glColor3f(0,0,0); drawTriangle(triangle);
25
26     Point s2[3], s05[3];
27     for(int i=0;i<3;i++){ s2[i].x=triangle[i].x*2; s2[i].y=triangle[i].y*2; }
28     for(int i=0;i<3;i++){ s05[i].x=triangle[i].x*0.5f; s05[i].y=triangle[i].y*0.5f; }
29
30     glColor3f(0,0,1); drawTriangle(s2);
31     glColor3f(1,0,1); drawTriangle(s05);
32
33     glFlush();
34 }
35
36 void init(){
37     glClearColor(1,1,1,1);
38     glMatrixMode(GL_PROJECTION); glLoadIdentity();
39     gluOrtho2D(-10,10,-10,10);
40 }
41
42 int main(int argc,char** argv){
43     glutInit(&argc,argv); glutInitWindowSize(800,800); glutCreateWindow("Scaling");
44     init(); glutDisplayFunc(display); glutMainLoop(); return 0;
45 }

```

Output:

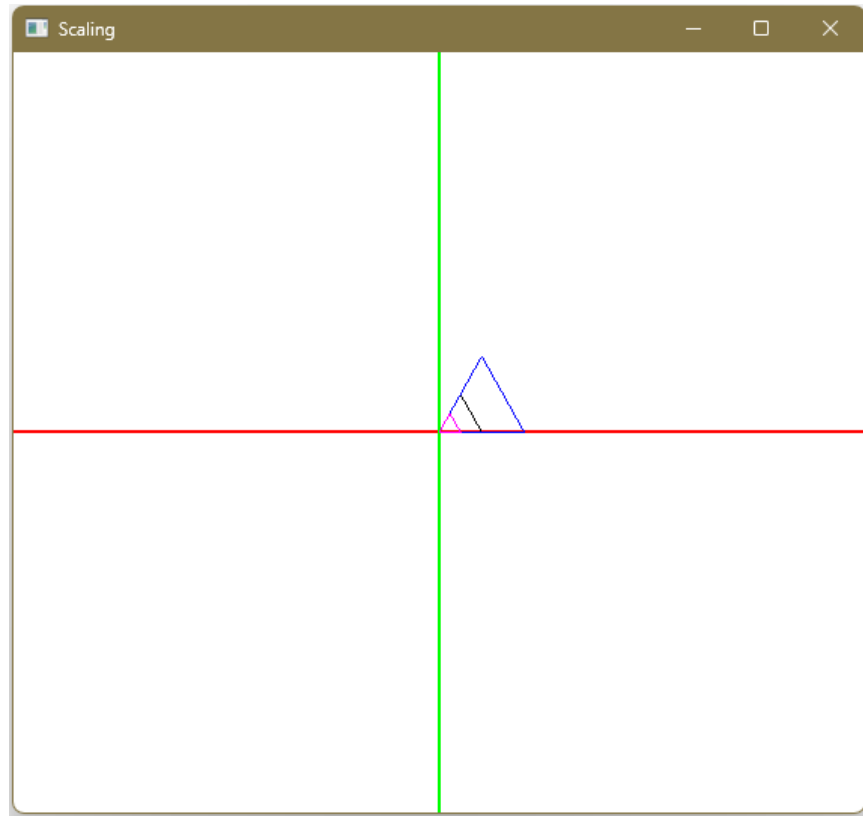


Figure 2: Output of the following code.

Code: Rotation ($\Theta = 45^\circ$) $\rightarrow (\Theta * \pi / 180)$

```
1 #include <GL/glut.h>
2 #include <cmath>
3
4 struct Point { float x, y; };
5 Point triangle[3]={{0,0},{1,0},{0.5f,1}};
6
7 void drawTriangle(Point pts[3]){
8     glBegin(GL_LINES);
9     for(int i=0;i<3;i++){ int n=(i+1)%3; glVertex2f(pts[i].x,pts[i].y); glVertex2f(pts[n].x,pts[n].y); }
10    glEnd();
11 }
12
13 void drawAxes(){
```

```

14  glLineWidth(2);
15  glColor3f(1,0,0); glBegin(GL_LINES); glVertex2f(-10,0); glVertex2f(10,0); glEnd();
16  glColor3f(0,1,0); glBegin(GL_LINES); glVertex2f(0,-10); glVertex2f(0,10); glEnd();
17  glLineWidth(1);
18 }
19
20 void display(){
21  glClear(GL_COLOR_BUFFER_BIT);
22  drawAxes();
23
24  glColor3f(0,0,0); drawTriangle(triangle);
25
26  float theta = 45.0f * 3.14159265f / 180.0f;
27  float c = cosf(theta), s = sinf(theta);
28  Point rot[3];
29  for(int i=0;i<3;i++){ rot[i].x = triangle[i].x*c - triangle[i].y*s; rot[i].y = triangle[i].x*s + triangle[i].y*c; }
30
31  glColor3f(0,0,1); drawTriangle(rot);
32
33  glFlush();
34 }
35
36 void init(){
37  glClearColor(1,1,1,1);
38  glMatrixMode(GL_PROJECTION); glLoadIdentity();
39  gluOrtho2D(-10,10,-10,10);
40 }
41
42 int main(int argc, char** argv){
43  glutInit(&argc,argv); glutInitWindowSize(800,800); glutCreateWindow("Rotation 45 deg CCW");
44  init(); glutDisplayFunc(display); glutMainLoop(); return 0;
45 }

```

Output:

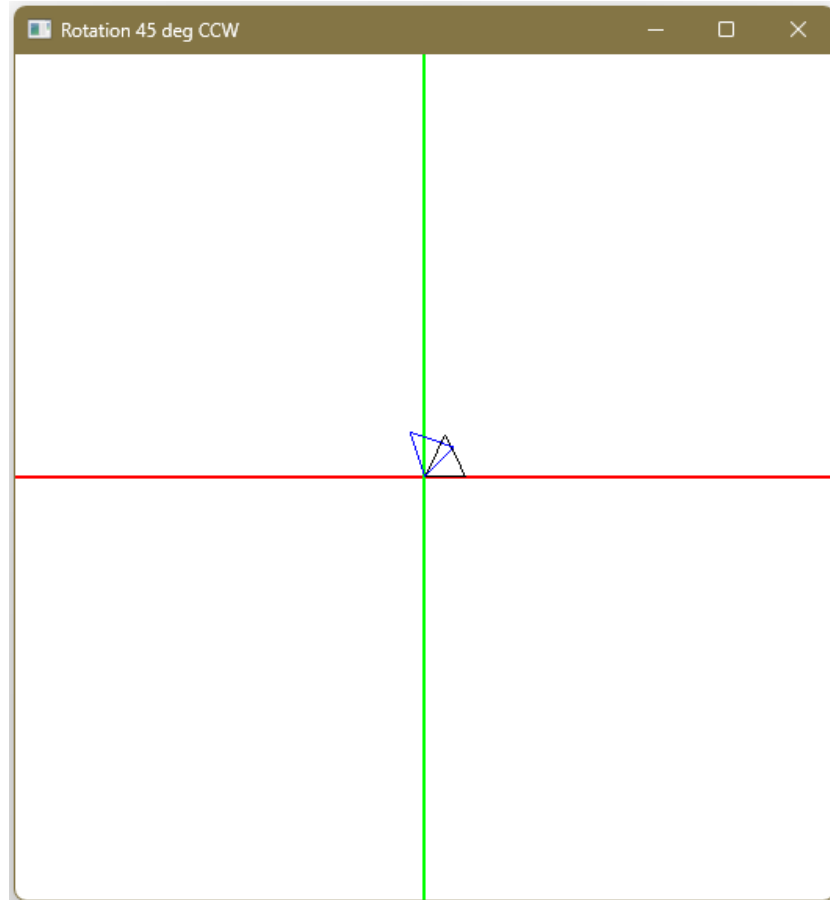


Figure 3: Output of the following code.

Code: Reflection about x and y-axis.

```

1  #include <GL/glut.h>
2
3  struct Point { float x, y; };
4  Point triangle[3]={0,0},{1,0},{0.5f,1}};
5
6  void drawTriangle(Point pts[3]){
7      glBegin(GL_LINES);
8      for(int i=0;i<3;i++){ int n=(i+1)%3; glVertex2f(pts[i].x,pts[i].y); glVertex2f(pts[n].x,pts[n].y); }
9      glEnd();
10 }
11
12 void drawAxes(){
13     glLineWidth(2);
14     glColor3f(1,0,0); glBegin(GL_LINES); glVertex2f(-10,0); glVertex2f(10,0); glEnd();

```

```

15  glColor3f(0,1,0); glBegin(GL_LINES); glVertex2f(0,-10); glVertex2f(0,10); glEnd();
16  glLineWidth(1);
17  }
18
19  void display(){
20      glClear(GL_COLOR_BUFFER_BIT);
21      drawAxes();
22
23      glColor3f(0,0,0); drawTriangle(triangle);
24
25      Point rx[3], ry[3];
26      for(int i=0;i<3;i++){ rx[i].x = triangle[i].x; rx[i].y = -triangle[i].y; }
27      for(int i=0;i<3;i++){ ry[i].x = -triangle[i].x; ry[i].y = triangle[i].y; }
28
29      glColor3f(0,0,1); drawTriangle(rx);
30      glColor3f(1,0,1); drawTriangle(ry);
31
32      glFlush();
33  }
34
35  void init(){
36      glClearColor(1,1,1,1);
37      glMatrixMode(GL_PROJECTION); glLoadIdentity();
38      gluOrtho2D(-10,10,-10,10);
39  }
40
41  int main(int argc, char** argv){
42      glutInit(&argc,argv); glutInitWindowSize(800,800); glutCreateWindow("Reflection");
43      init(); glutDisplayFunc(display); glutMainLoop(); return 0;
44  }

```

Output:

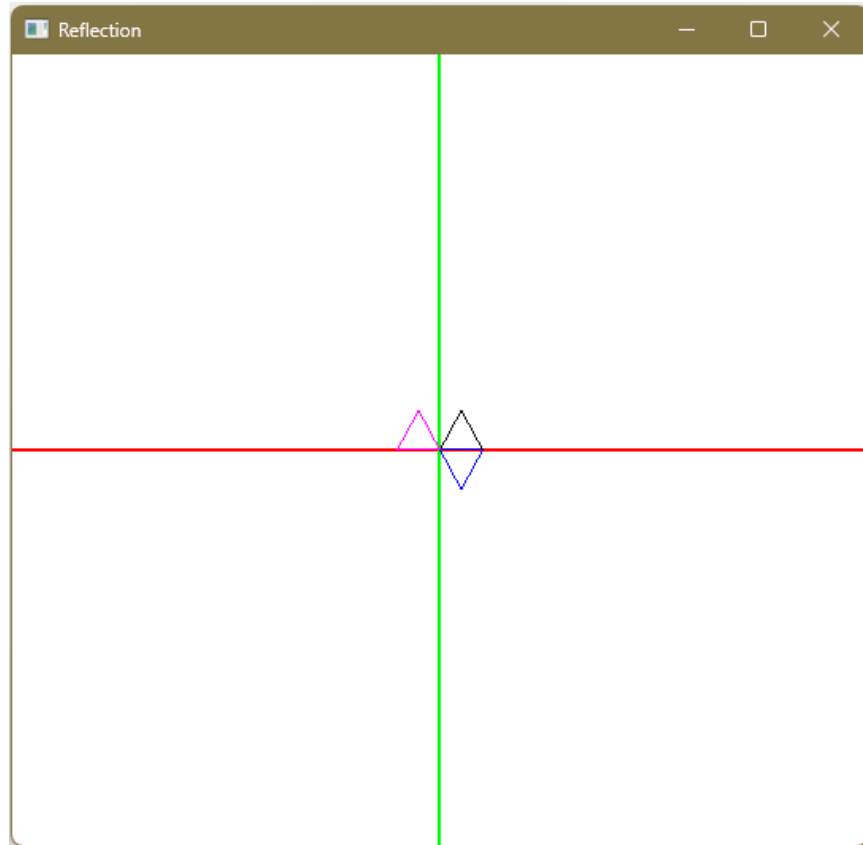


Figure 4: Output of the following code.

Code: Shearing about x and y-axis (Shr = 2).

```

1  #include <GL/glut.h>
2
3  struct Point { float x, y; };
4  Point triangle[3]={0,0},{1,0},{0.5f,1}};
5
6  void drawTriangle(Point pts[3]){
7      glBegin(GL_LINES);
8      for(int i=0;i<3;i++){ int n=(i+1)%3; glVertex2f(pts[i].x,pts[i].y); glVertex2f(pts[n].x,pts[n].y); }
9      glEnd();
10 }
11
12 void drawAxes(){
13     glLineWidth(2);
14     glColor3f(1,0,0); glBegin(GL_LINES); glVertex2f(-10,0); glVertex2f(10,0); glEnd();
15     glColor3f(0,1,0); glBegin(GL_LINES); glVertex2f(0,-10); glVertex2f(0,10); glEnd();

```

```

16  glLineWidth(1);
17 }
18
19 void display(){
20  glClear(GL_COLOR_BUFFER_BIT);
21  drawAxes();
22
23  glColor3f(0,0,0); drawTriangle(triangle);
24
25  float shx = 2.0f;
26  Point sh[3];
27  for(int i=0;i<3;i++){ sh[i].x = triangle[i].x + shx*triangle[i].y; sh[i].y = triangle[i].y; }
28
29  glColor3f(0,0,1); drawTriangle(sh);
30
31  glFlush();
32 }
33
34 void init(){
35  glClearColor(1,1,1,1);
36  glMatrixMode(GL_PROJECTION); glLoadIdentity();
37  gluOrtho2D(-10,10,-10,10);
38 }
39
40 int main(int argc,char** argv){
41  glutInit(&argc,argv); glutInitWindowSize(800,800); glutCreateWindow("Shearing");
42  init(); glutDisplayFunc(display); glutMainLoop(); return 0;
43 }

```

Output:

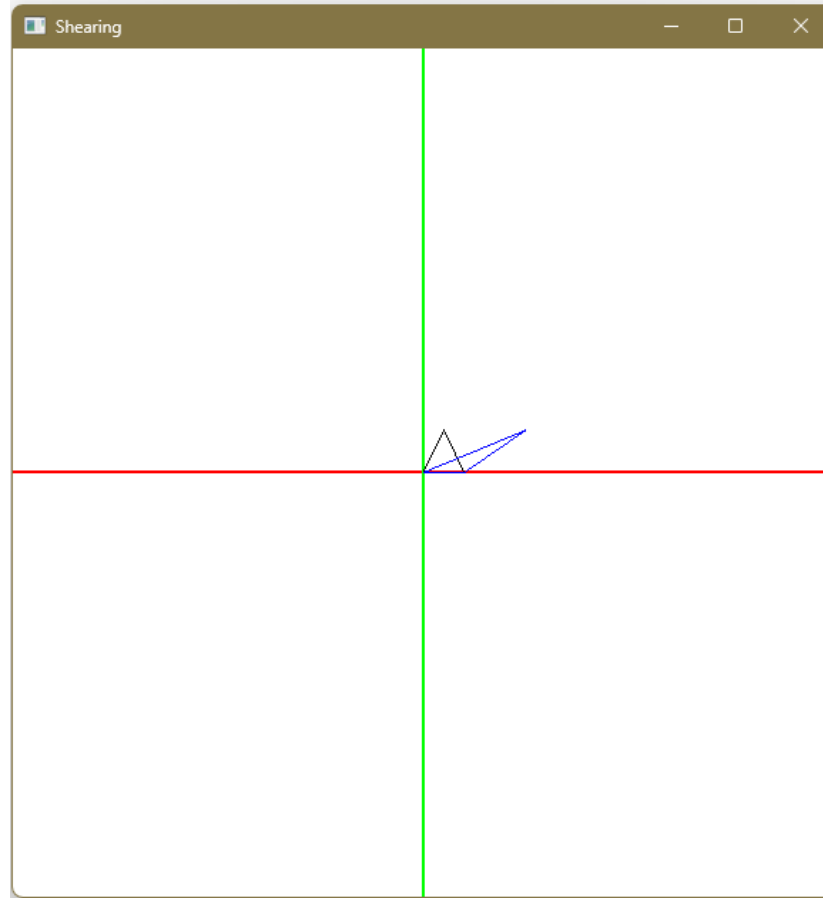


Figure 5: Output of the following code.

Conclusion:

The experiment on 2D transformations for a triangle was successfully carried out. All the transformations --- translation, scaling, rotation, reflection, and shearing --- worked as expected. The results showed how each operation affects the shape and position of the triangle, helping to understand the geometric principles behind graphical transformations.